

MÓDULO 2
Arquitecturas de seguridad
y técnicas
Ciberseguridad

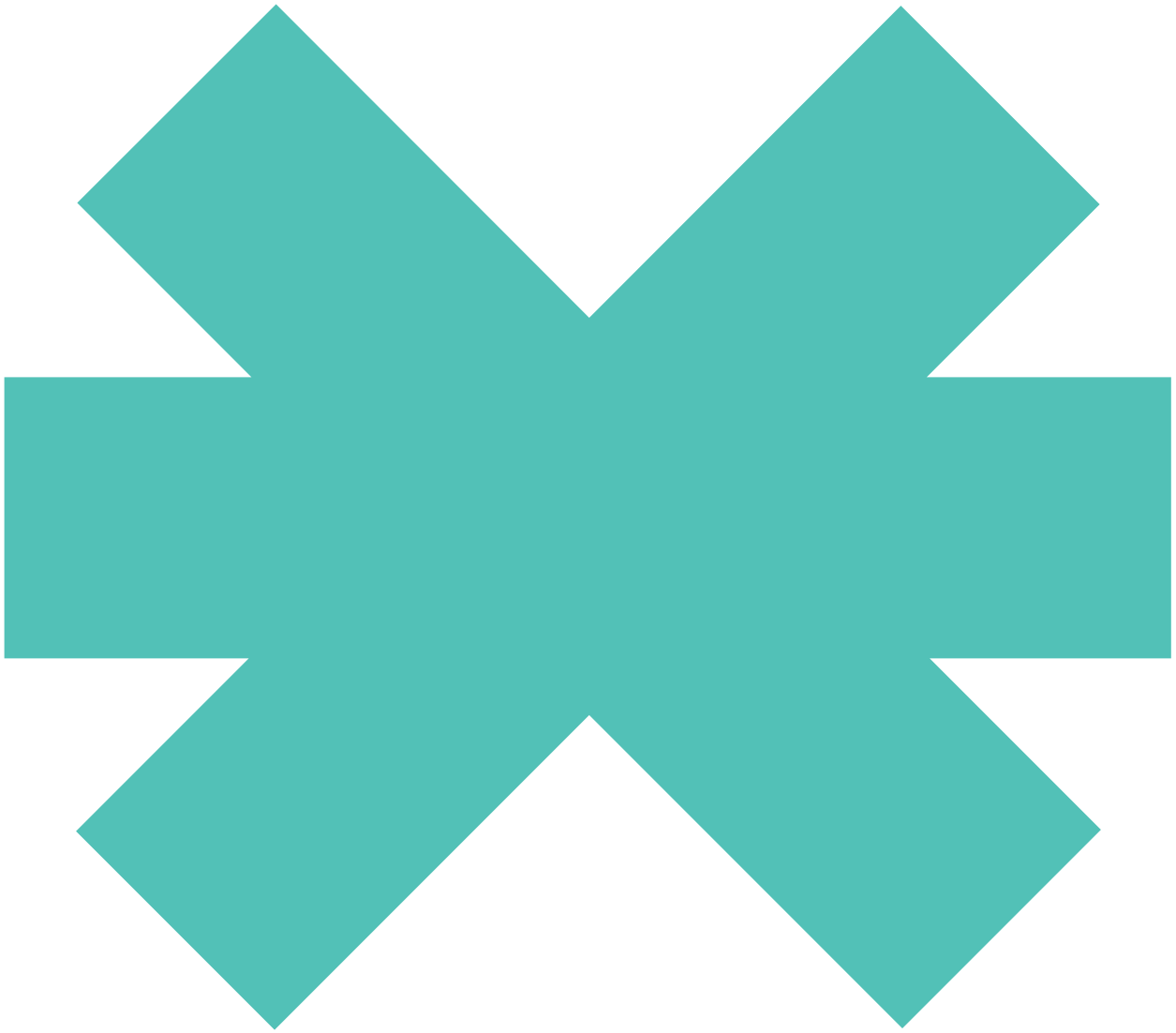
5

Seguridad en dispositivos móviles y web



New
Technology
School

Tokio.



5 Seguridad en dispositivos móviles y web

Sumario

5.1	Seguridad en dispositivos móviles	158
5.2	Android	165
5.3	iOS	167
5.4	Seguridad en aplicaciones web	169

5.1 Seguridad en dispositivos móviles

Los dispositivos móviles han experimentado un crecimiento casi exponencial en los últimos años con la introducción de los *smartphones* y su capacidad para conectarse a internet, lo que los convierte en un objetivo muy interesante para los atacantes, ya que podrían controlarse desde cualquier lugar del mundo. Adicionalmente, en los países más desarrollados, resulta muy habitual que la población cuente con dos dispositivos, el personal y el corporativo, o bien que se combinen ambas esferas en uno solo, contribuyendo a que se pueda exfiltrar información de la compañía por posibles errores en la configuración del dispositivo por parte del usuario/a.

Partiendo de este contexto, los medios de comunicación informan a diario sobre ataques de diferentes criticidades cuyo objetivo son los dispositivos móviles.

Información almacenada

Los teléfonos móviles almacenan una ingente cantidad de información que puede resultar relevante para resolver incidencias, investigaciones o ataques. Así, durante una investigación forense, estos dispositivos pueden revelar varios tipos de datos:

- Historial de llamadas y lista de contactos.
- Información compartida en aplicaciones de mensajería instantánea.
- Datos en diferentes formatos multimedia, como vídeos, imágenes y mensajes de voz.
- Contraseñas.
- Información sobre las aplicaciones instaladas.
- Archivos del sistema, registros y mensajes de error.
- Archivos borrados.
- Geolocalización del dispositivo.

Como podemos observar, todos estos datos pueden proporcionar numerosas pistas y pruebas en el transcurso de una investigación.

Parte de esta información se hallará en la memoria interna del dispositivo (**información no volátil**), mientras que otra parte se localizará en la memoria extraíble de este (**información volátil**).

MEMORIA INTERNA

Denominamos memoria a la capacidad de almacenamiento que presenta el dispositivo y que se destina a albergar el sistema operativo, así como todas las aplicaciones instaladas. Representa la capacidad total de almacenamiento disponible en el dispositivo para guardar datos, aplicaciones, archivos y otros contenidos.

LA TARJETA SIM

La tarjeta SIM (*Subscriber Identity Module*) es un chip integrado en una tarjeta de plástico cuya función consiste en almacenar de manera segura la clave de servicio del usuario o código PIN, que lo identifica en la red móvil de un operador telefónico. De este modo, el operador puede reconocer a cada uno de sus usuarios/as y habilitar el acceso al número de teléfono y a los servicios que ofrece.

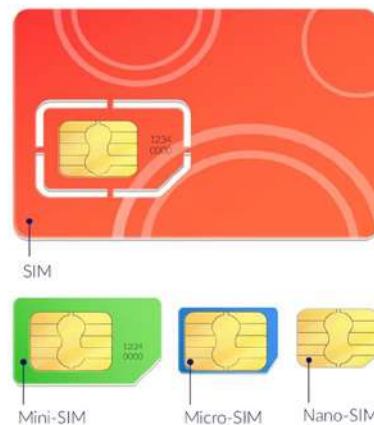


Figura 11. Tarjetas SIM. Fuente: Freepik.com

LA MEMORIA EXTERNA

Abarca los dispositivos de almacenamiento que se utilizan en un dispositivo móvil, pero que no forman parte inherente de él. Uno de los tipos más conocidos de memoria externa es la **tarjeta micro SD** (*Secure Digital*), que es extraíble y se utiliza para ampliar el almacenamiento de la memoria interna del dispositivo, permitiendo albergar diversos tipos de información digital, como programas y archivos. Si bien al principio las tarjetas SD poseían capacidades de almacenamiento de 32 MB o 64 MB, hoy en día se pueden encontrar de hasta 1 TB.

Vulnerabilidades en dispositivos móviles

Es importante comprender las vulnerabilidades que presentan los dispositivos móviles y familiarizarnos con ellas, pues en numerosas ocasiones, los incidentes se derivan de ellas. Por tanto, identificar estas vulnerabilidades puede resultar de gran ayuda para mantener seguros nuestros dispositivos y también en el caso de que se deba realizar un análisis forense.

- **Vulnerabilidades en las aplicaciones.** Teniendo en cuenta el tiempo limitado que suele dedicarse al desarrollo de aplicaciones móviles y el uso frecuente de herramientas de código abierto, estas aplicaciones pueden contener numerosas vulnerabilidades. Además, en muchos casos, la prioridad es obtener un gran número de descargas en lugar de garantizar la seguridad. Además, en entornos corporativos, el control sobre los dispositivos móviles suele ser menor que el que se lleva a cabo en los equipos de escritorio, lo que incrementa la probabilidad de que los empleados y las empleadas descarguen aplicaciones no autorizadas.
- **Vulnerabilidades en el sistema operativo.** Los sistemas operativos de los dispositivos móviles también pueden presentar vulnerabilidades que han de considerarse cuando se distribuyan entre los empleados y las empleadas de una organización. Cabe destacar que, generalmente, los dispositivos Android demandan un período de parcheo de vulnerabilidades más prolongado que los dispositivos iOS.

- **Vulnerabilidades en la red.** Las vulnerabilidades de la interfaz de red de un dispositivo móvil suelen aprovecharse a menudo, ya que, por su propia naturaleza, este requiere una conexión y, en ocasiones, se conecta a redes potencialmente peligrosas careciendo del mismo nivel de seguridad que los ordenadores o servidores.
- **Vulnerabilidades en la web y en el contenido.** Este tipo de vulnerabilidades se centran en explotar el contenido de páginas web, vídeos y fotos para comprometer la seguridad de las aplicaciones y los sistemas operativos, obteniendo así un acceso no autorizado.

Los ciberdelincuentes suelen aprovechar este tipo de vulnerabilidades recurriendo a técnicas de ingeniería social para engañar a los usuarios y obtener credenciales de acceso. A tal efecto, crean aplicaciones con contenido malicioso que el usuario puede descargarse de tiendas no oficiales, interfieren en la comunicación entre usuarios en una red wifi pública mediante un ataque MitM, acceden a dispositivos IoT que se encuentran vinculados al móvil, jaquean un dispositivo robado o aprovechan las ventanas de exposición que puedan presentar aquellos móviles que no se encuentren actualizados.

Ataques más comunes

Aunque en el módulo anterior ya nos hemos ocupado de los principales tipos de ataques cibernéticos, ahora nos referiremos a aquellos que se encuentran expresamente dirigidos al entorno móvil:

- **Smishing.** Se trata de un ataque específico para teléfonos móviles donde el atacante envía un mensaje de texto suplantando a una persona o a una entidad legítima y solicitando al usuario que clique sobre un enlace o que se ponga en contacto con un número de teléfono.

Cuando incorpora un enlace, suele conducir a una URL de apariencia similar a la de la organización que se trata de suplantar para, a través de un formulario, acceder a los datos que introduzca el usuario. En cambio, en el supuesto de solicitar un contacto vía WhatsApp, el atacante pretende hacerse pasar por un conocido y, apelando a su falta de liquidez, solicitar un pago a través de Bizum.

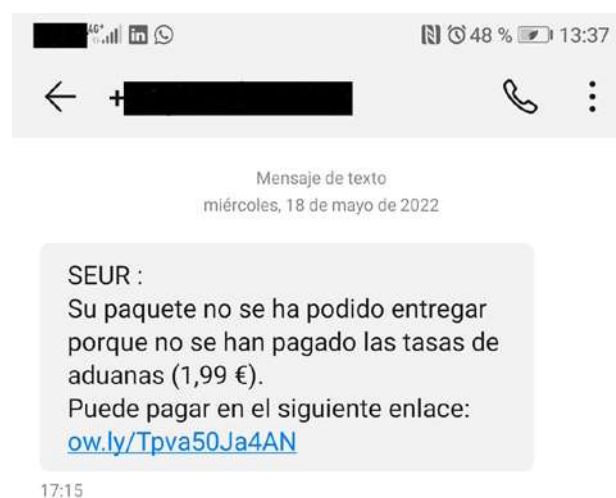


Figura 12. Smishing

- **Overlay.** *Malware* que se ejecuta en segundo plano sin conocimiento del usuario. En el momento en el que este abre una aplicación vinculada al *malware*, se activa una superposición de pantalla que imita la interfaz de la aplicación original, pero que ha sido diseñada para capturar información confidencial. Normalmente, en el listado del *malware*, figuran aplicaciones bancarias.
- **Ataque wifi de tipo WEP.** Ataque que consiste en aprovechar las debilidades presentes en el protocolo de cifrado, de manera que cuando el usuario se conecta a una red wifi que posee el algoritmo de cifrado WEP, permanece expuesto a que un atacante descifre la clave, comprometiendo el sistema.
- **Ataque wifi de tipo WPA.** El algoritmo de cifrado WPA presenta algunas mejoras frente a su predecesor WEP; sin embargo, también puede ser susceptible de ataques por fuerza bruta dirigidos a descifrar la clave de la red. Por este motivo, el algoritmo se mejoró a WPA2 y, posteriormente, a WPA3, que es el más seguro.
- **Bluejacking.** Técnica que consiste en enviar mensajes o datos no solicitados a dispositivos cercanos a través de conexiones Bluetooth. Aunque no siempre resulta peligrosa, en ocasiones, puede emplearse para desplegar *malware* por medio de publicidad invasiva.
- **Bluebugging.** En este caso, el atacante explota una vulnerabilidad en el protocolo Bluetooth para acceder al dispositivo objetivo y, una vez establecida la conexión, puede robar datos, enviar archivos, realizar llamadas, etc. Se trata de un método de espionaje muy utilizado.
- **Replay attack.** En este caso, el atacante intercepta la comunicación entre dos dispositivos para grabar la secuencia de datos enviados entre ambas partes, como una transacción financiera o una autenticación de usuario. Posteriormente, el atacante reproduce esos datos para engañar al sistema que los recibe. La técnica más conocida a tal efecto es el MitM.
- **RF jamming (Radio Frequency jamming).** Ataque que consiste en interferir o bloquear intencionadamente las señales de radiofrecuencia para interrumpir la comunicación inalámbrica normal.
- **Clickjacking.** También conocido como ataque de compensación UI, consiste en engañar al usuario que entra a un sitio web y clics sobre un elemento oculto o engañoso. En este sentido, se superponen o enmascaran elementos visibles de un sitio web con elementos ocultos, de manera que el usuario, al interactuar con lo que considera un sitio legítimo, en realidad ejecuta un código malicioso o bien es redirigido por medio de un falso formulario para robar sus credenciales de acceso.

Malware en móviles

Como hemos visto hasta ahora, los dispositivos móviles no se encuentran exentos de sufrir un ataque por parte de los ciberdelincuentes. Según su funcionalidad, podemos distinguir varios tipos de *malware* relacionados con estos dispositivos que, en esencia, son similares al *malware* en general:

- **Malware bancario.** El *malware* orientado a aplicaciones bancarias va en aumento, especialmente aquellos troyanos dirigidos a usurpar las credenciales de los usuarios y las usuarias. Además, con frecuencia, se aplican técnicas de ingeniería social.
- **Ransomware móvil.** Como ya hemos mencionado, este tipo de *malware* bloquea o cifra los ficheros del dispositivo. Son más conocidos los que se detectan en los PC por su facilidad para extenderse dentro de una red corporativa.

- **Spyware móvil.** Su objetivo es supervisar la actividad del dispositivo, registrar su ubicación y sustraer información crítica, como nombres de usuario y contraseñas de cuentas de correo electrónico o de sitios de comercio electrónico.
- **Malware de MMS.** Los atacantes también tratan de aprovechar la comunicación basada en contenido multimedia para distribuir *malware* e infectar dispositivos móviles.
- **Adware móvil.** Este tipo de *malware*, que ya cuenta con un largo historial, también implica riesgos, ya que, además de resultar bastante molesto, puede afectar al rendimiento del dispositivo, compromete nuestra privacidad, reúne gran cantidad de datos y puede descargar contenido malicioso si clicamos donde no debemos.
- **Troyanos de SMS.** También es frecuente que los cibercriminales recurran al contenido más popular de los teléfonos móviles, los mensajes de texto, para infectar los dispositivos.

Introducción al análisis forense en dispositivos móviles

Como hemos señalado al principio de este tema, los dispositivos móviles pueden representar una importante fuente de información, que podemos clasificar en volátil y no volátil, siendo la información volátil muy susceptible de perderse si no se siguen los procedimientos adecuados.

Antes de continuar, esclarezcamos otro concepto importante en el ámbito del análisis forense, el artefacto. Denominamos **artefacto** a cualquier indicio, evidencia o rastro que deja una actividad o evento en un sistema o dispositivo digital, tratándose de un elemento fundamental en el análisis forense. Entre los artefactos más comunes de cara al análisis digital, se encuentran los siguientes:

- **Registro de llamadas.** Permite listar todas las llamadas realizadas, recibidas o canceladas, así como su duración y la hora a la que se registraron, lo que puede resultar interesante para determinar los contactos y las interacciones del usuario o usuaria.
- **Mensajes de texto y mensajes instantáneos.** Todos los mensajes de texto y las conversaciones instantáneas pueden aportar información relevante como contactos, fechas, horas y contenido adjunto de interés.
- **Descargas y archivos multimedia.** Fotos, vídeos, archivos de audio y documentos descargados. Este tipo de archivos también puede contener información relevante, como los intereses, la ubicación, la fecha y la hora de la captura, etc.
- **Historial de navegación y actividad en internet.** Los navegadores web almacenan un historial donde pueden localizarse los lugares visitados, las búsquedas realizadas, los datos ingresados en un formulario, etc.
- **Correo electrónico.** Asimismo, los correos electrónicos recibidos y enviados pueden aportar información significativa como contactos, fechas y horas de envío, datos confidenciales, credenciales de acceso, etc.
- **Aplicaciones y datos de redes sociales.** Las aplicaciones instaladas y las redes sociales también pueden almacenar información relevante como publicaciones, listas de contactos, teléfonos, etc.

- **Datos de ubicación.** Los dispositivos móviles suelen registrar los datos de su ubicación, tanto por GPS como por torres de telefonía o por las redes wifi a las que se han conectado.
- **Registro de eventos del sistema.** El sistema operativo del dispositivo almacena un registro de todos los eventos, errores, modificaciones en la configuración, etc.

En numerosas ocasiones, el análisis forense de un dispositivo móvil representa un importante desafío. En este sentido, para extraer información de los artefactos correctamente, han de valorarse diversos factores:

- **Diferencias de *hardware*,** dada la gran variedad de dispositivos móviles disponibles en el mercado.
- **Sistemas operativos móviles.** Aunque Android e iOS son los predominantes, existen otros en el mercado.
- **Funciones de seguridad de la plataforma móvil,** como mecanismos de cifrado.
- **Escasez de recursos.** La gran variedad de dispositivos determina que resulte necesario contar con las herramientas necesarias para cada uno de ellos.
- **Estado genérico del dispositivo.** Debe procederse de forma diferente en función del estado del dispositivo para evitar la pérdida de información.
- **Técnicas antiforenses,** como la eliminación o falsificación de datos.
- **Naturaleza dinámica de la evidencia.** En ocasiones, el mero hecho de acceder a una evidencia ya puede alterar su estado al almacenar una información adicional.
- **Restablecimiento accidental.** Podría darse el caso de restablecer el sistema por error, lo que ocasionaría la pérdida de toda la información.
- **Alteración de dispositivos.** Mover o renombrar archivos, así como modificar aplicaciones instaladas.
- **Recuperación del código de acceso.** El código PIN, el PUK o el patrón de desbloqueo han de ser averiguados por el analista si se desconocen, quien también ha de romper la barrera de seguridad biométrica si se encuentra activada.
- **Protección de comunicación.** Será necesario usar papel de aluminio, bolsas de Faraday o bien cualquier tipo de envoltorio que pueda impedir las comunicaciones con una red externa.
- **Programas maliciosos** presentes en el dispositivo, que podrían propagarse a otros dispositivos.
- **Problemas legales** en los que pueda encontrarse involucrado el dispositivo.

Una vez identificados los factores anteriores, resulta necesario valorar el **proceso de extracción de pruebas**, que puede llevarse a cabo de distintas formas, pues no se ha establecido un procedimiento estándar. En cualquier caso, el aspecto más relevante consiste, sin duda, en resultar sumamente precisos, documentando todo el proceso y siguiendo un orden adecuado en cada una de las acciones que se emprendan:

- Ingesta de pruebas.
- Identificación.

- Preparación.
- Aislamiento.
- Procesamiento.
- Verificación:
 - Comparación de los datos extraídos con los datos del teléfono.
 - Emplear múltiples herramientas y contrastar los resultados.
 - Usar valores *hash*.
- Documentación y presentación de informes.
- Presentación.
- Archivo.

Más adelante, profundizaremos en el análisis forense digital y nos aproximaremos a los diferentes procedimientos y métodos a los que se puede recurrir para llevarlo a cabo con éxito.

Consejos generales

Una vez identificadas las diferentes vulnerabilidades inherentes a los dispositivos móviles, así como los principales ataques o *malware* que pueden afectarles, cabe destacar una serie de recomendaciones de cara a su protección y seguridad:

- **Utilizar un wifi seguro.** Como ya hemos señalado, las redes wifi públicas pueden suponer un punto de acceso a nuestro dispositivo si nos conectamos a ellas, de manera que debemos evitar siempre que resulte posible su uso y recurrir, principalmente, a nuestra red personal o corporativa.
- **Supervisar nuestro correo electrónico.** Asimismo, el correo electrónico representa uno de los puntos de entrada a nuestro dispositivo más habituales para los ciberdelincuentes.
- **Ser coherentes.** Es decir, reflexionar antes de pulsar en algún enlace, ya que detrás puede existir una amenaza.
- **Evitar descargas automáticas** (WhatsApp, Telegram, etc.), especialmente si desconocemos quién remite el mensaje.
- **Instalar un antivirus.**
- **No liberar ni acceder a la raíz del dispositivo**, esto es, no hacer *root* al usuario del dispositivo (*jailbreak* en el caso de iOS).

5.2 Android

Android es el sistema operativo móvil más extendido a nivel mundial. Desarrollado por Google y basado en el *kernel* de Linux, se emplea principalmente en dispositivos móviles y tabletas, si bien también se ha expandido a otros dispositivos, como televisores y relojes inteligentes o componentes de automóviles.

Una de las características distintivas de Android es su naturaleza de código abierto, lo que significa que el código fuente del sistema operativo permanece accesible y puede ser modificado y personalizado por los fabricantes de dispositivos y los desarrolladores, de ahí su popularidad. Esta característica ha conducido a que se conforme una amplia comunidad de desarrolladores y a la creación de una significativa variedad de aplicaciones.

A partir de la versión Android 5.0 (Lollipop), el entorno de ejecución Dalvik fue reemplazado por el entorno de ejecución ART (Android Runtime), que utiliza una máquina virtual de registro basada en *bytecode*, diferenciándose de Dalvik en que realiza una compilación anticipada (AOT, *Ahead-of-Time*) en lugar de una compilación *Just-in-Time* (JIT). De este modo, el código de la aplicación se compila antes de su ejecución, mejorando el rendimiento y la eficiencia energética del dispositivo.

Desde la primera versión de Android, que se lanzó en 2005, cuando Google adquirió la compañía, han aparecido más de veinte. La última versión, Android 14, estará disponible a lo largo de 2023.

Arquitectura de Android

Android se basa en una estructura de cinco capas que permite el funcionamiento del sistema operativo y las aplicaciones en dispositivos móviles. A continuación, para comprender su funcionamiento, describiremos brevemente cada una de dichas capas:

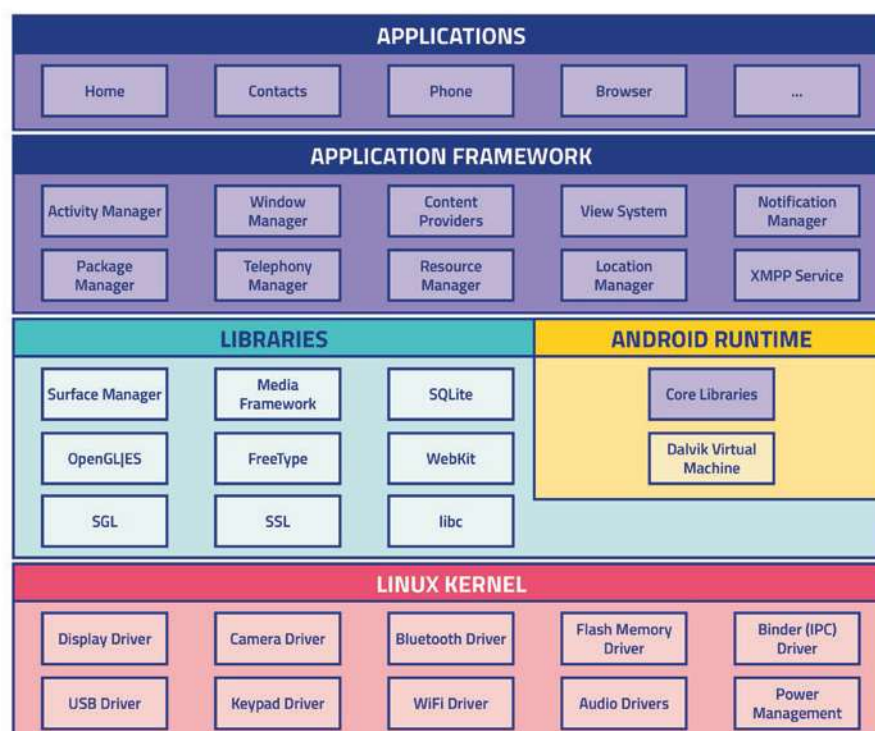


Figura 13. Estructura de Android

- **Núcleo del sistema operativo.** Android depende de Linux para los servicios base del sistema, como seguridad, gestión de memoria, gestión de procesos, pila de red y modelo de controladores.
- **Entorno de ejecución (Android runtime).** Android incluye un set de bibliotecas base que proporcionan la mayor parte de las funciones esenciales para las aplicaciones, como gráficos, acceso a bases de datos, conectividad de red y manejo de archivos. Además, se encarga de compilar y ejecutar las aplicaciones de Android.
- **Marco de trabajo de aplicaciones (application framework).** Proporciona un conjunto de componentes y servicios que facilitan el desarrollo de aplicaciones. Los desarrolladores cuentan con acceso completo a las mismas API del entorno de trabajo que usan las aplicaciones base. La arquitectura ha sido diseñada para simplificar la reutilización de componentes, de modo que una aplicación puede publicar sus capacidades y, posteriormente, cualquier otra aplicación puede recurrir a ellas.
- **Bibliotecas.** Android incluye un conjunto de bibliotecas de C/C++ que emplean varios componentes del sistema. Estas características se exponen a los desarrolladores a través del marco de trabajo de aplicaciones de Android.
- **Aplicaciones preinstaladas en el sistema operativo.** Entre ellas, el navegador web, la aplicación de mensajes, el cliente de correo electrónico y otras de carácter básico.

Android Studio

Android Studio es el entorno oficial de cara al desarrollo de las aplicaciones de Android que ha reemplazado a Eclipse, donde anteriormente se creaban tales aplicaciones por medio de un *plugin* específicamente preparado para ello. La primera versión se lanzó en 2014 y se basa en el *software* de IntelliJ IDEA de JetBrains. Además, permite desarrollar aplicaciones tanto en Java como en Kotlin, que es el lenguaje más extendido ahora mismo, sustituyendo de manera paulatina a Java. En la siguiente imagen, se muestra un ejemplo de Android Studio.

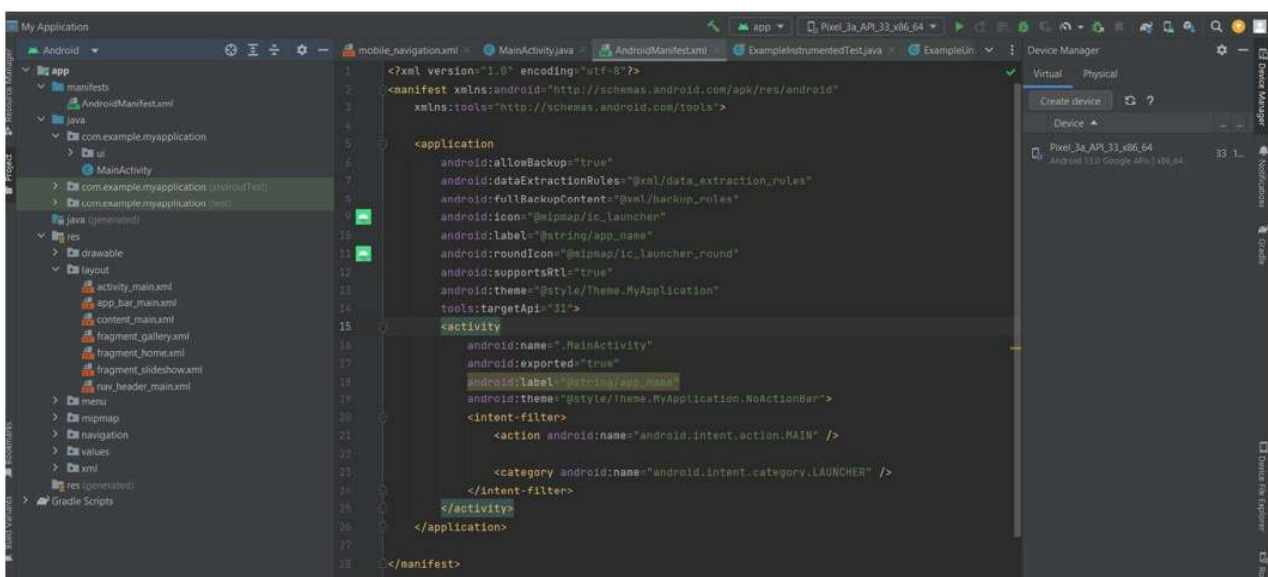


Figura 14. Interfaz de Android Studio

5.3 iOS

iOS es un sistema operativo de Apple desarrollado exclusivamente para sus dispositivos móviles iPhone, iPad y iPod Touch. La primera versión, iOS 1, vio la luz en 2007, dos años después que Android, y presentaba funcionalidades básicas como llamadas, mensajes, navegador web y aplicaciones preinstaladas. Hasta el momento, se han lanzado un total de 16 versiones, siendo iOS 16 la última de ellas.

Arquitectura de iOS

La arquitectura de iOS se basa en el sistema Unix y es ligeramente diferente a la de Android, ya que posee cuatro capas en lugar de cinco.

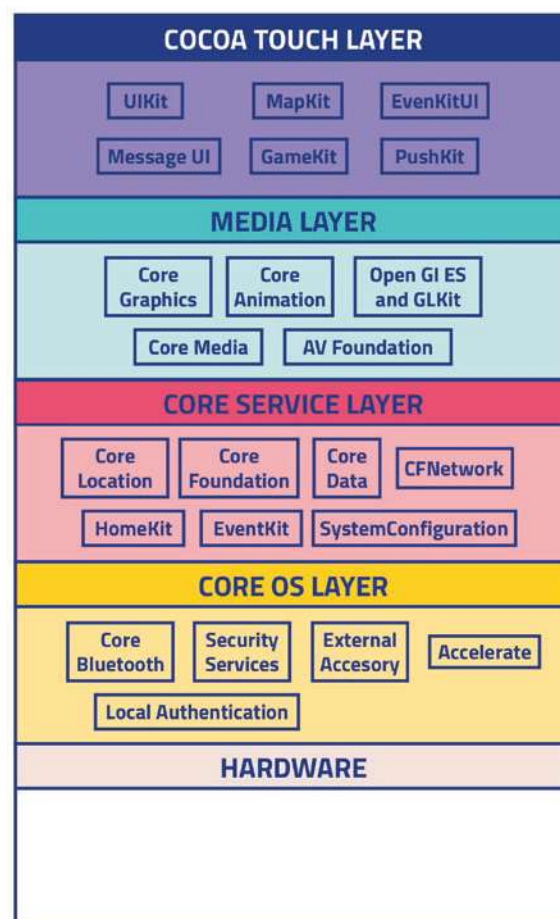


Figura 15. Estructura de iOS

- **Capa del núcleo del sistema operativo (Core OS).** Proporciona los componentes básicos del sistema operativo, como el *kernel*/XNU, controladores de dispositivos, seguridad y servicios de bajo nivel.
- **Capa de servicios del sistema (Core Services).** Incluye servicios compartidos y *frameworks* que utilizan las aplicaciones en iOS. Los principales servicios son *Core Foundation* (manejo de datos y colecciones), *Core Location* (servicios de ubicación), *Core Motion* (sensores y movimiento) y *Core Telephony* (servicios de telecomunicaciones).

- **Capa de servicios de medios (*Media Services*)**. Provee servicios relacionados con el contenido multimedia, por ejemplo, audio, vídeo, animaciones y gráficos. Incluye *AVFoundation* (manejo de audio y vídeo), *Core Animation* (animación en la interfaz de usuario), *Core Audio* (manejo de audio), *Core Graphics* (manejo de gráficos), *Core Image* (procesamiento de imagen), etc.
- **Capa de *frameworks* de alto nivel (*Cocoa Touch*)**. Es la capa más alta de la estructura de datos de iPhone y comprende los *frameworks* de uso más comunes para los desarrolladores y desarrolladoras de aplicaciones. Se encarga de la interfaz de usuario y de la interacción con este. Incluye *UIKit* (interfaz de usuario), *MapKit* (mapas y ubicación), *EventKit* (calendario y eventos), *HealthKit* (salud y actividad), etc.

XCode

XCode es el entorno de desarrollo integrado de Apple, destinado a la creación de aplicaciones para los sistemas operativos iOS y macOS. Este IDE proporciona a los desarrolladores y las desarrolladoras las herramientas necesarias para escribir código, diseñar interfaces de usuario, depurar, probar y distribuir aplicaciones.

El lenguaje de programación que se emplea en este entorno es **Swift**, específicamente creado por Apple como una alternativa al Objective-C para el desarrollo de aplicaciones en sus sistemas operativos.

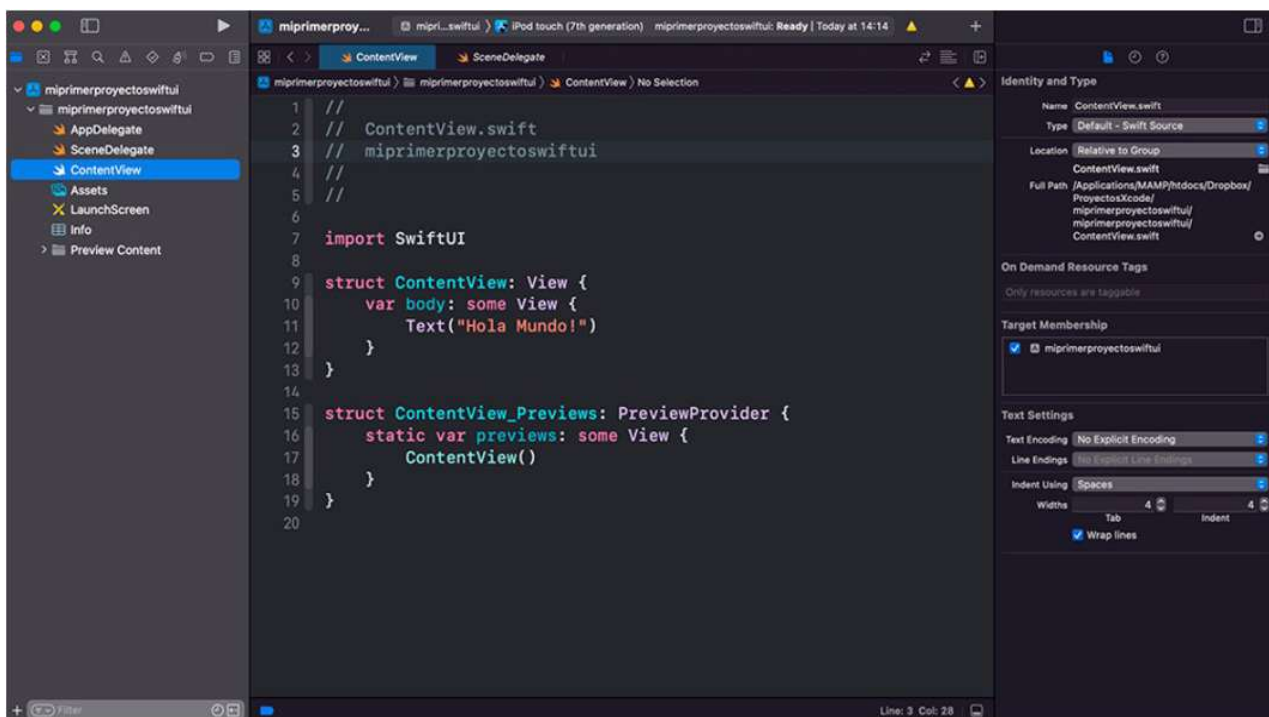


Figura 16. Interfaz de XCode

5.4 Seguridad en aplicaciones web

Otro de los objetivos más habituales de los atacantes son las páginas webs, donde se han de tener en cuenta todo tipo de medidas para eludir las amenazas, además de considerar una serie de conceptos que analizaremos a lo largo de este punto.

Resulta importante señalar que su seguridad ha mejorado notablemente durante los últimos años gracias a los *firewalls* e IDS (Sistemas de Detección de Intrusos), contribuyendo a que la parte servidora permanezca más protegida. En este sentido, la parte que se despliega en el propio navegador es la que queda menos resguardada, de manera que, a la hora de abordar el diseño de una web, es necesario realizar un buen análisis para protegerla de todo tipo de ataques.

En esta línea, además de referirnos a aquellos conceptos básicos asociados a las aplicaciones web y a los servidores o a definiciones absolutamente necesarias como la de HTTP, explicaremos cómo utilizar algunas herramientas que permitan al usuario final realizar *hacking* a través de buscadores.

Hacking con buscadores web

En esta sección, nos detendremos en cómo aprovechar las ventajas que ofrecen los buscadores para obtener información de diversas páginas web, así como para identificar posibles configuraciones erróneas. Cabe destacar que todas estas búsquedas se llevan a cabo en la red de internet y que son completamente legales, ya que se efectúan mediante buscadores reconocidos como Google, Bing y Shodan o a través de los propios robots de Google. También exploraremos la automatización de búsquedas recurriendo a *dorks* y, por último, profundizaremos en el concepto de metadatos.

En primer lugar, debemos subrayar las diferencias existentes entre dos conceptos íntimamente relacionados, los motores de búsqueda y los navegadores web. Los **motores de búsqueda** son las **aplicaciones o programas en línea que ejecutan las búsquedas en internet**, de modo que su función consiste en indexar y catalogar las páginas web recurriendo a complejos algoritmos de clasificación para que los usuarios y las usuarias puedan localizar la información que procuran. Los **navegadores web**, por su parte, son el *software* que permite a los usuarios y las usuarias acceder al contenido de internet, visualizarlo y navegar por él, es decir, actúan como **intermediarios entre el usuario y los sitios web**, pues interpretan el código HTML y el de otros lenguajes de programación empleados para diseñar y mostrar las páginas web.

A continuación, nos centraremos en los motores de búsqueda Google, Shodan y Bing.

- **Google** es uno de los motores de búsqueda más populares y utilizados del mundo. Fundado en 1998, cuenta con un algoritmo extremadamente sofisticado que indexa una gran cantidad de páginas web y ofrece resultados de búsquedas precisos y relevantes. Además, Google ofrece una amplia gama de servicios y productos como aplicaciones de productividad o herramientas para desarrolladores y desarrolladoras.
- **Bing**, también conocido como Live Search, es otro motor de búsqueda líder, propiedad de la empresa Microsoft, que fue lanzado en 2009. Actualmente, ha alcanzado una amplia cuota de mercado y, al igual que su competidor Google, cuenta con un poderoso algoritmo para indexar contenido.

- **Shodan**, a diferencia de los dos motores anteriores, presenta un enfoque muy diferente. En lugar de rastrear sitios webs convencionales, se especializa en localizar dispositivos conectados a internet. De esta forma, rastrea servidores, cámaras web, dispositivos IoT, sistemas de control industrial, etc., lo que permite a los usuarios y las usuarias reunir datos sobre estos dispositivos, como su configuración, sus vulnerabilidades y otros detalles que podrían contener información sensible.

Los navegadores más conocidos son Firefox, Chrome, Explorer, Edge, Ópera, Safari... y los utilizaremos de cara a la correcta comprensión de los motores de búsqueda mencionados.



Figura 17. Iconos de diferentes navegadores web

HACKING CON GOOGLE: Robots.txt

Para abordar el *hacking* con Google nos ocuparemos, en primer lugar, de los archivos “robots.txt”. Básicamente, se trata de un fichero de texto que se utiliza para indicar las rutas a las que pueden acceder los buscadores, evitando que el propio servidor se sobrecargue. De esta forma, se controla el rastreo y la indexación de contenido en un sitio web y se impide que ciertas páginas o directorios sean indexados en los resultados de búsqueda. Resulta importante destacar que estos ficheros no sirven para que no se indexe en un servidor de búsqueda, para ello habría que definirlo como *noindex* o disponer de una contraseña en la página web.

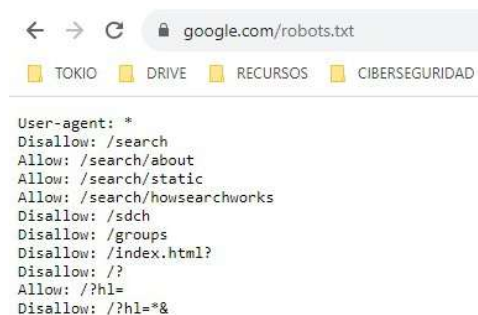


Figura 18. Ejemplo del archivo robots.txt

Como podemos suponer, si el motor de búsqueda cuenta con la posibilidad de acceder a esta información, un usuario también, de manera que podría revelar rutas de la web que, *a priori*, se desconocían.

HACKING CON GOOGLE: dorks

Técnica que consiste en realizar una búsqueda avanzada en las propias búsquedas de Google y que, habitualmente, utilizan periodistas e investigadores/as o incluso en el ámbito de la ciberseguridad. Esta técnica permite localizar información específica a la que no se podría acceder fácilmente mediante una búsqueda tradicional.

De esta manera, es posible localizar ficheros, diferentes activos e incluso puertos abiertos que podrían provocar fugas de información. El tipo de contenido que se puede encontrar mediante este tipo de búsquedas posee un considerable valor para todo tipo de perfiles (tanto malintencionados como bienintencionados), por ejemplo:

- Información sobre personas/organizaciones.
- Contraseñas.
- Documentos confidenciales.
- Versiones de servicios vulnerables.
- Directorios expuestos.

El procedimiento para utilizar esta técnica es muy sencillo, simplemente debemos dirigirnos a la barra del navegador y usar los siguientes comandos:

Operador	Utilidad	Ejemplo
""	Búsqueda con coincidencia exacta.	"github"
site:	Busca un sitio web específico.	site: github.com
filetype:	Busca resultados que posean la extensión de archivo indicada en cualquier lugar de la página web.	filetype:pdf
ext:	Busca resultados que posean la extensión del archivo especificada, pero solo se aplica al final de la URL de la página web.	ext:pdf guía de usuario
inurl:	Busca la palabra especificada en una URL.	inurl:hacking
intext:	Resultados de páginas en cuyo contenido aparezca la palabra especificada.	intext:hacking
allinurl:	Busca todas las palabras especificadas en una URL.	allinurl:Google hacking
allitext:	Muestra resultados de páginas en cuyo contenido aparecen las palabras especificadas.	allitext:Google hacking
allintitle:	Muestra resultados de páginas en cuyo título aparecen todas las palabras especificadas.	allintitle:Google hacking
-	Símbolo de exclusión. Se excluye de los resultados todo lo que se sitúe a continuación del símbolo.	hacking -Google

*	Símbolo de comodín. El asterisco puede sustituirse por cualquier palabra.	site: *.ejemplo.com
cache:	Muestra la versión del caché de la web.	cache:Google.com
OR	Operador lógico, también representado por	ext:pdf OR ext:txt
AND	Operador lógico	Google AND Bing

Veamos algunos ejemplos que podrían usarse:

- Buscar servidores FTP expuestos: `intitle:"index of" inurl:ftp`
- Buscar servicios que corran en el puerto 8080: `inurl:8080 -intext:8080`

Como curiosidad, señalar que existe una base de datos donde la comunidad va incorporando diferentes formas de detectar este tipo de vulnerabilidades; se llama GHDB (Google Hacking DataBase, <https://www.exploit-db.com/google-hacking-database>) y podría resultar muy útil para efectuar pruebas sobre diferentes servidores.

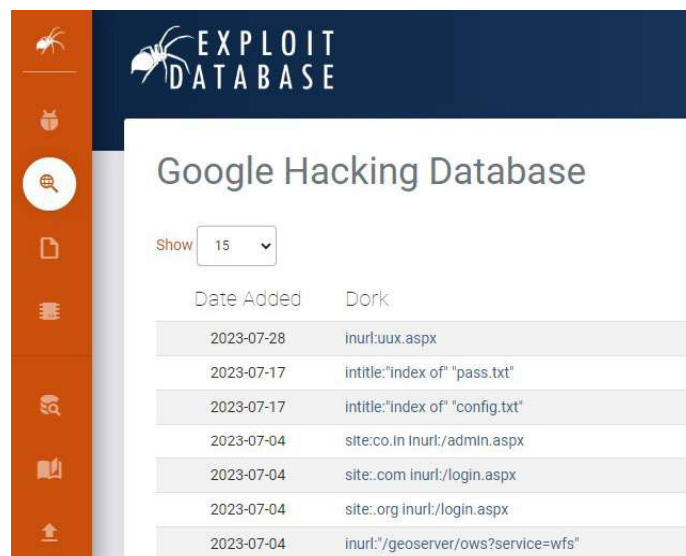


Figura 19. Captura de pantalla de GHDB

HACKING CON BING

Bing ofrece la posibilidad de realizar el mismo tipo de búsquedas que Google y los resultados son similares, pero no idénticos, dado que cada motor podría haber almacenado una información diferente y proporcionar más datos acerca del sitio web que se esté analizando.

Algunos ejemplos de *dorks* con Bing serían:

- **<>>>**. Busca el contenido exacto que se sitúe entre las comillas.
- **AND, OR, NOT**. Son operadores lógicos (sin valor).

- **filetype**. Busca el tipo de fichero especificado.
- **contains**. Contiene enlaces a un tipo de fichero determinado.
- **site**. Busca un dominio o «extensión de dominio» concreto (dominio.com o simplemente .com).
- **loc**. Determina el país donde queremos realizar la búsqueda (códigos de los países).
- **intitle, inbody, inanchor**. Se emplean en la búsqueda de un título, un cuerpo de página o textos de enlace, respectivamente.
- **prefer**. Se emplea para asignar importancia al contenido especificado.
- **feed**. Busca *feeds* de páginas web.
- **findfromdomain**. Busca todos los sitios que cuenten con un link al dominio especificado.
- **Inurl**. En Bing no funciona como en Google; sin embargo, en cualquier búsqueda que realicemos, lo aplicará por defecto.

Asimismo, se puede recurrir a ciertas herramientas gratuitas (*open source*) que podrían facilitar el uso de este tipo de comandos, como [bing-dorks](#) · [GitHub Topics](#) · [GitHub](#).

HACKING CON SHODAN

Shodan es un motor de búsqueda que también nos permite localizar información en la red, pero no se trata de un buscador de páginas webs, sino de máquinas conectadas a internet; por tanto, si se realiza una búsqueda de un lugar, Shodan mostrará máquinas conectadas y/o relacionadas con este.

Como hemos señalado al principio del tema, Shodan es completamente legal, ya que busca en información pública, lo que no implica, lógicamente, acceder a dichos servidores.

Shodan resulta muy útil para auditores de seguridad informática, ya que permite buscar por IP, geolocalización o incluso por información del país al que pertenece dicho servicio. Paralelamente, proporciona datos que permiten determinar hasta qué punto se encuentra expuesto el servidor para evitar que algún agente maligno pueda acceder a él.

Para utilizar Shodan, simplemente hay que acceder a la web y buscar información relativa a un servicio, IP, etc.

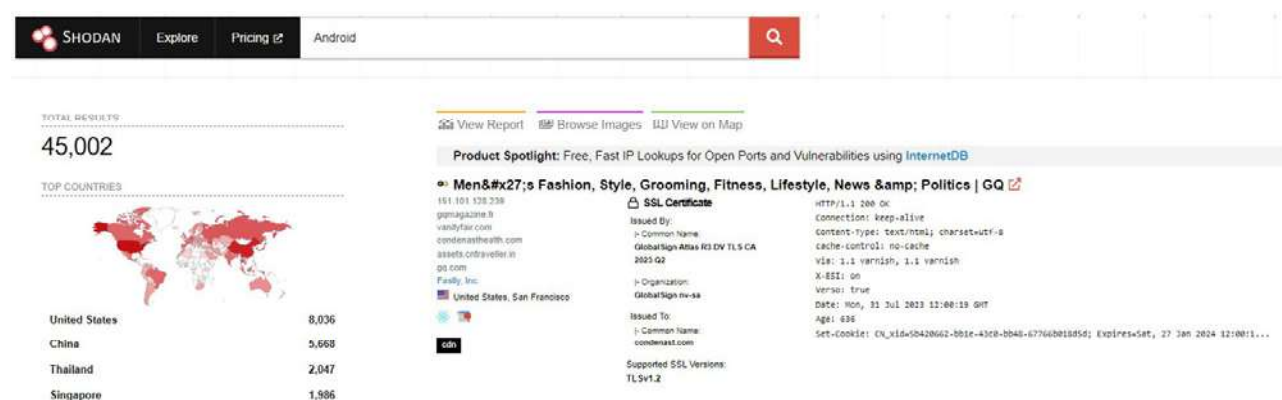


Figura 20. Búsqueda en Shodan

HTTP y HTTPS

HTTP es el protocolo base que se utiliza para transferir datos entre un navegador web y un servidor web. Se trata de un protocolo en texto plano, por lo que carece de cualquier tipo de seguridad, y tampoco posee estado, de manera que cada petición es independiente y no se tiene en cuenta si se ha efectuado otra previamente.

Si bien resulta útil para transferir información básica y pública, como páginas web estáticas, no proporciona una capa de seguridad para proteger los datos confidenciales que puedan ser intercambiados, entre ellos, contraseñas, números de tarjeta de crédito, etc.

En cambio, HTTPS es una extensión del protocolo anterior, donde la “S” indica que la conexión entre el navegador y el servidor se encuentra cifrada y, por tanto, protegida. En este sentido, se utiliza un protocolo de seguridad adicional denominado SSL (*Secure Socket Layer*) o su sucesor, TLS (*Transport Layer Security*).

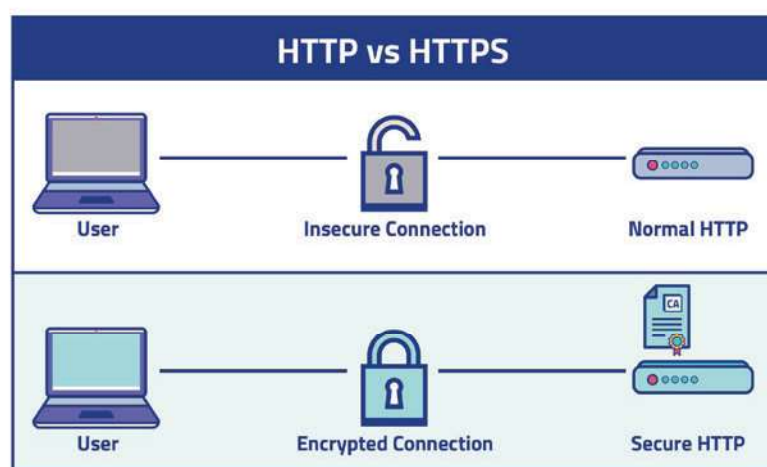


Figura 21

Servidor web

Un servidor web es un contenedor donde se van a desplegar las páginas que, posteriormente, serán visibles desde el navegador. Por tanto, su función consiste en desplegar el *backend* que recibirá las peticiones desde el navegador, es decir, en almacenar, procesar y entregar contenido a los usuarios y las usuarias cuando acceden a un sitio web.

Cuando un usuario ingresa una URL en el navegador o clics sobre un enlace, el navegador envía una solicitud al servidor web asociado a dicha URL, que la procesa, localiza los recursos solicitados y los envía de vuelta al navegador del usuario en forma de una página web.

API

Una API (*Application Programming Interface*) es un conjunto de reglas y protocolos que permiten que diferentes aplicaciones y sistemas se comuniquen y compartan información entre sí a través de internet. Se trata, pues, de una forma estandarizada de intercambio de datos que posibilita que dos aplicaciones se conecten y se comuniquen de manera segura y eficiente.

Las API se utilizan ampliamente de cara al desarrollo de aplicaciones y servicios en línea, ya que permiten que diferentes componentes y sistemas trabajen conjuntamente y de manera integrada sin necesidad de conocer los detalles internos de cada uno. Así, en lugar de acceder directamente a la base de datos o a los recursos de otra aplicación, se puede interactuar con ella a través de su API.

Lenguajes de programación web

Uno de los principales aspectos que debemos abordar a la hora de tratar las aplicaciones web son los lenguajes de programación, pues nos permitirán entender cómo los navegadores traducen el código a las interfaces que muestran:

- **HTML (*Hyper Text Markup Language*)**. Lenguaje de marcas que se emplea para la construcción y diseño de las páginas web y que posibilitan a los equipos de desarrollo estructurar su contenido para que pueda ser interpretado por el navegador.

```
<> EJEMPLO.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Página Web Básica</title>
5  </head>
6  <body>
7
8    <h1>Bienvenido a Mi Página Web</h1>
9
10   <p>Esta es una página web básica creada con HTML.</p>
11
12   <p>Puedes añadir más contenido aquí, como imágenes, listas, enlaces y mucho más.</p>
13
14   <h2>Un ejemplo de lista:</h2>
15   <ul>
16     <li>Elemento 1</li>
17     <li>Elemento 2</li>
18     <li>Elemento 3</li>
19   </ul>
20
21   <h2>Un ejemplo de enlace:</h2>
22   <a href="https://www.ejemplo.com">Visitar Ejemplo.com</a>
23
24 </body>
25 </html>
```

Figura 22. Lenguaje HTML

- **CSS (*Cascading Style Sheets*)**. Se trata de un lenguaje de estilos que proporciona el formato con el que se va a mostrar la página web. Los equipos de desarrollo lo emplean para controlar la apariencia visual de la página (estilo de texto, color, imágenes, tamaños, etc.).

```
# EJEMPLO.css > ...
1  /* Estilos generales */
2  body {
3      font-family: Arial, sans-serif;
4      line-height: 1.6;
5      margin: 20px;
6      padding: 0;
7  }
8  h1 {
9      color: #333;
10 }
11 p {
12     color: #666;
13 }
14 /* Estilos para la lista */
15 ul {
16     list-style-type: square;
17     color: #009688;
18 }
19 /* Estilos para el enlace */
20 a {
21     color: #2196F3;
22     text-decoration: none;
23 }
24 a:hover {
25     text-decoration: underline;
26 }
27
```

Figura 23. Lenguaje CSS

- **JavaScript**. Lenguaje de programación de alto nivel, interpretado y orientado a objetos mediante el cual los desarrolladores y las desarrolladoras pueden manipular el contenido de la página, responder a eventos, realizar validaciones, crear animaciones y efectos visuales o interactuar con las API y los servicios web.

```
JS EJEMPLO.js > ...
1  function validarFormulario() {
2
3      var nombre = document.getElementById("nombre").value;
4      var email = document.getElementById("email").value;
5      var mensaje = document.getElementById("mensaje").value;
6
7      if (nombre === "" || email === "" || mensaje === "") {
8          alert("Por favor, complete todos los campos.");
9          return false;
10     }
11
12     var emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
13     if (!emailRegex.test(email)) {
14         alert("Por favor, ingrese un correo electrónico válido.");
15         return false;
16     }
17
18     alert("Formulario enviado correctamente. ¡Gracias!");
19     return true;
20 }
21
```

Figura 24. Lenguaje JavaScript

- **PHP.** Lenguaje de programación del lado del servidor. También se trata de un lenguaje interpretado cuya función principal es generar contenido dinámico en las páginas web y trabajar con las bases de datos para almacenar y recuperar información.

```

EJEMPLO.php
1  <?php
2  $servername = "localhost";
3  $username = "usuario";
4  $password = "contraseña";
5  $dbname = "nombre_base_de_datos";
6
7  $conn = new mysqli($servername, $username, $password, $dbname);
8
9  if ($conn->connect_error) {
10     die("Error de conexión: " . $conn->connect_error);
11 }
12
13 $sql = "SELECT id, nombre, email FROM usuarios";
14 $result = $conn->query($sql);
15
16 if ($result->num_rows > 0) {
17     while($row = $result->fetch_assoc()) {
18         echo "ID: " . $row["id"] . ", Nombre: " . $row["nombre"] . ", Email: " . $row["email"] . "<br>";
19     }
20 } else {
21     echo "No se encontraron resultados.";
22 }
23
24 $conn->close();
25
26 ?>

```

Figura 25. Lenguaje PHP

Metodologías existentes

Se puede recurrir a numerosas metodologías para analizar la seguridad de las páginas webs, desde OSSTM (*Open Source Security Testing Methodology Manual*), que mide el detalle técnico de seguridad durante todo el proceso, es decir, antes, después y a lo largo del desarrollo, hasta PTES (*Penetration Testing Execution Standard*), que proporciona un lenguaje común para todos los profesionales, junto con evaluaciones de seguridad. Ahora bien, la más conocida es OWASP (*Open Web Application Security Project*).

OWASP TOP 10

OWASP es un proyecto abierto cuyo objetivo es definir **diez reglas clave de cara a la seguridad** en diferentes ámbitos, como aplicaciones móviles, web, servidores, etc.

A continuación, analizaremos las diez reglas más comunes de acuerdo con la última versión, correspondiente al año 2021:

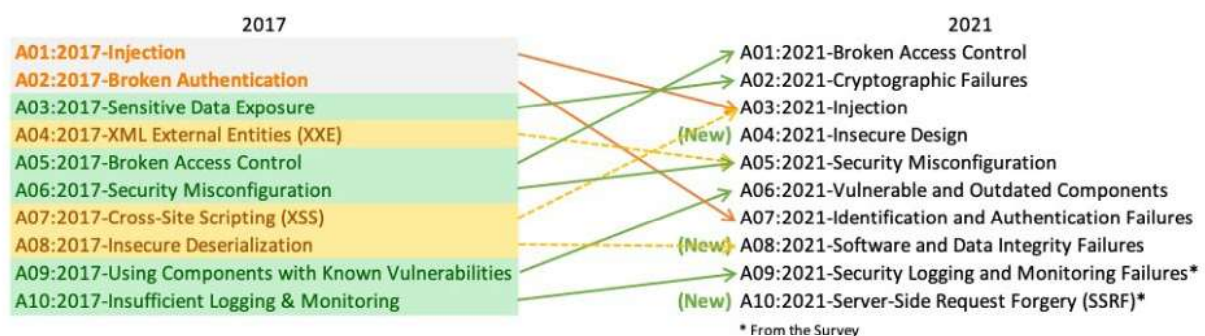


Figura 26. TOP 10 OWASP

- **A01:2021 – Pérdida de control de acceso.** Sube desde la quinta posición hasta la categoría que implica mayor riesgo para la seguridad de las aplicaciones web. Los datos obtenidos indican que, como promedio, el 3,81 % de las aplicaciones probadas presentan una o más *Common Weakness Enumerations* (CWE), con más de 318 000 ocurrencias de CWE en dicha categoría. Además, las 34 CWE relacionadas con la pérdida de control de acceso registraron más apariciones en las aplicaciones que el resto.
- **A02:2021 – Fallos criptográficos.** Antes conocida como A3:2017 - *Exposición de datos sensibles*, que representaba una característica más que una causa raíz, sube una posición, ubicándose en la segunda. El nuevo nombre se centra en las deficiencias relacionadas con la criptografía y, frecuentemente, conlleva la exposición de datos confidenciales o el compromiso del sistema.
- **A03:2021 – Inyección.** Desciende hasta la tercera posición. El 94 % de las aplicaciones fueron probadas con algún tipo de inyección y registraron una tasa de incidencia máxima del 19 %, con un promedio del 3.37 %; asimismo, las 33 CWE relacionadas con esta categoría poseen la segunda mayor tasa de incidencia en las aplicaciones, con 274 000 ocurrencias. En esta edición, el *Cross-Site Scripting* se incorpora a esta categoría de riesgo.
- **A04:2021 – Diseño inseguro.** Nueva categoría creada para la edición 2021 y orientada a los riesgos relacionados con las deficiencias de diseño. Si realmente queremos madurar como industria, debemos “mover a la izquierda” el proceso de desarrollo de las actividades de seguridad, es decir, necesitamos más modelos de amenazas, patrones y principios con diseños seguros y arquitecturas de referencia. Un diseño inseguro no puede enmendarse con una implementación perfecta, dado que, por definición, los controles de seguridad requeridos nunca se crearon para defenderse de ataques específicos.
- **A05:2021 - Configuración de seguridad incorrecta.** Ascende desde la sexta posición que ocupaba en la edición anterior, lo que no resulta sorprendente, dado que el *software* altamente configurable ha incrementado su presencia. Con el objeto de detectar algún tipo de configuración incorrecta, se probaron el 90 % de las aplicaciones, registrándose una tasa de incidencia promedio del 4,5 % y más de 208 000 casos de CWE relacionadas con esta categoría, a la que se ha incorporado en esta edición la A4:2017-*Entidades externas XML (XXE)*.
- **A06:2021 – Componentes vulnerables y desactualizados.** Anteriormente denominada *Uso de componentes con vulnerabilidades conocidas*, esta categoría ocupa el segundo lugar en el Top 10 de la encuesta a la comunidad, si bien también obtuvo datos suficientes para incorporarse al Top 10 a través del análisis de datos. Desde la edición de 2017, asciende desde la novena posición, tratándose de un problema conocido que resulta difícil probar, así como evaluar su riesgo. Se trata de la única categoría que no cuenta con ninguna CVE relacionada con las CWE incluidas, de manera que se considera una vulnerabilidad predeterminada con ponderaciones de impacto de 5,0.
- **A07:2021 – Deficiencias de identificación y autenticación.** Previamente denominada *Pérdida de autenticación*, esta categoría desciende desde la segunda posición y ahora abarca aquellas CWE específicamente relacionadas con los defectos de identificación. Si bien sigue siendo una parte integral del Top 10, la mayor disponibilidad de *frameworks* estandarizados parece estar mitigando su incidencia.
- **A08:2021 - Errores en el *software* y en la integridad de los datos.** Se trata de una nueva categoría incluida en la edición de 2021 y centrada en realizar estimaciones acerca de las actualizaciones de *software*, los datos críticos y los *pipelines* CI/CD sin verificación de integridad. Constituye uno de los factores de mayor

impacto según los sistemas de ponderación de vulnerabilidades (CVE/CVSS, siglas en inglés para *Common Vulnerability and Exposures/Common Vulnerability Scoring System*). En esta edición, la A8:2017-*Deserialización insegura* se incorpora a esta extensa categoría de riesgo.

- **A09:2021 – Fallos en el registro y el monitoreo.** Previamente denominada A10:2017-*Registro y monitoreo insuficientes*, se incorpora desde el Top 10 de la encuesta a la comunidad (tercer lugar), ascendiendo desde la décima posición de la edición anterior. Paralelamente, esta categoría se amplía para incluir más tipos de deficiencias que, si bien resultan difíciles de probar y no se encuentran bien representadas en los datos de CVE/CVSS, pueden afectar directamente a la visibilidad, a las alertas de incidentes y a los análisis forenses.
- **A10:2021 - Falsificación de solicitudes del lado del servidor.** También se incorpora desde el Top 10 de la encuesta a la comunidad, donde ocupa el primer lugar. Los datos muestran una tasa de incidencia relativamente baja con una cobertura de pruebas y unas calificaciones por encima del promedio para la capacidad de explotación e impacto. Esta categoría representa un escenario que cobra cada vez mayor importancia desde la perspectiva de la seguridad, aunque los datos todavía no lo reflejen.

Ataques más comunes

Existen numerosos métodos para llevar a cabo un ataque a aplicaciones webs; veamos los más comunes:

- **XSS (Cross Site Scripting).** Se trata de una secuencia de comandos cuyo propósito es inyectar algún tipo de código JavaScript malicioso en la página web. Podemos distinguir dos tipos de XSS:
 - **Almacenado.** Es el que conlleva mayor riesgo, pues consiste en insertar el código en la base de datos, de manera que se ejecutará siempre que se cargue la página, pudiendo revelar cualquier tipo de información o bloqueándola.
 - **Reflejado.** No resulta tan peligroso, pero permite ejecutar código JS dentro del navegador, de modo que podría revelar aquella información que se esté almacenando dentro de la propia página.

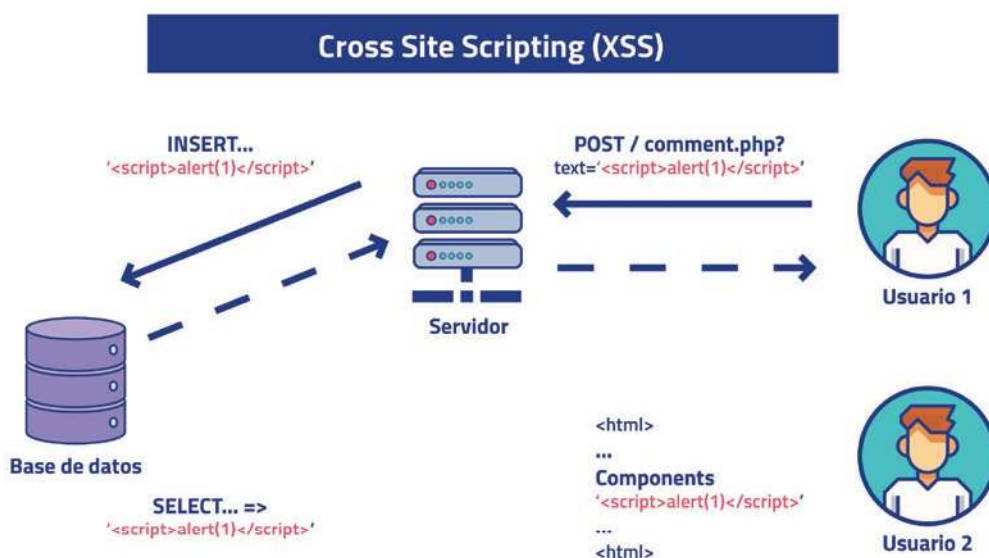


Figura 27

- **SQLi.** La posibilidad de realizar consultas no permitidas a través de la página web podría revelar información confidencial de la base de datos, de modo que resulta muy peligroso confiar en los datos de entrada de un usuario.

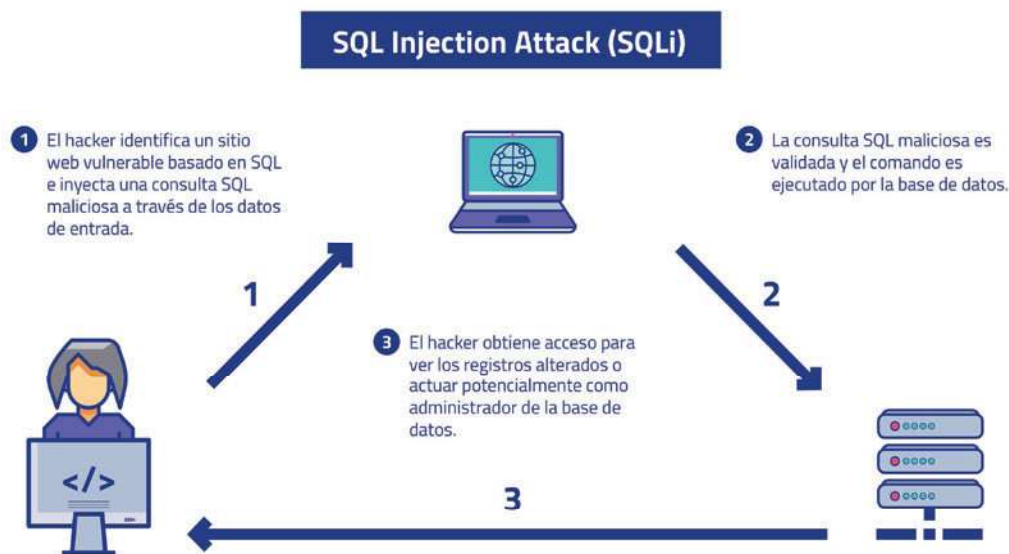


Figura 28

- **Fuerza bruta.** Las contraseñas representan las llaves del reino y son el mecanismo para la autenticación más utilizado y fácil de administrar; no obstante, en nombre de la usabilidad, se utilizan incorrectamente por parte de los usuarios y las usuarias. Así, lamentablemente, resulta muy común que muchas contraseñas sigan siendo 123456, *password* o *qwerty*.
- **Inyección de comandos (*command injection*).** El atacante aprovecha una vulnerabilidad de una aplicación web que posibilita la ejecución de comandos del sistema operativo, lo que normalmente resulta factible cuando esta no valida adecuadamente la entrada del usuario y permite que datos no confiables o maliciosos se interpreten como comandos válidos del sistema.
- **CSRF (*Cross-Site Request Forgery*).** También conocido como “ataque de falsificación de petición en sitios cruzados”, fuerza al usuario a ejecutar acciones no requeridas sobre una aplicación web donde se está autenticado. De este modo, por medio de ingeniería social, un atacante puede forzar a los usuarios y las usuarias a ejecutar determinadas acciones.



Figura 29

Herramientas de análisis

En el mercado, se encuentran disponibles numerosas herramientas destinadas a realizar análisis exhaustivos de páginas web. Entre ellas, podemos distinguir aquellas que demandan una licencia de pago y aquellas que son de código abierto (*open source*). Cada una de estas categorías ofrece diferentes soluciones para evaluar el rendimiento, la seguridad y la calidad de un sitio web.

Cabe señalar que el uso de estas herramientas **requiere del consentimiento del propietario o del administrador de los sistemas y de las redes que se van a escanear**.

NESSUS

Se trata de una herramienta que realiza escaneos de seguridad automatizados buscando vulnerabilidades conocidas en sistemas operativos, aplicaciones y dispositivos de red. Nessus realiza pruebas exhaustivas y ofrece informes detallados sobre las vulnerabilidades detectadas, clasificándolas en función de su gravedad y proporcionando recomendaciones de cara a su corrección.

Aunque se puede descargar una versión de prueba, la herramienta requiere, principalmente, una licencia comercial. El proceso se puede resumir del siguiente modo:

- **Escáner de puertos.** El módulo Nessusd ejecuta un análisis sobre los dispositivos conectados a una red y los puertos que se encuentren abiertos.
- **Detección de servicios.** Nessusd identifica los servicios de las aplicaciones que utilizan dichos puertos.
- **Identificación de vulnerabilidades.** Nessus Client compara la información obtenida con sus extensas bases de datos y señala las coincidencias con fallos de seguridad.
- **Sondeo final.** El sistema comprueba que no se incluyan falsos positivos dentro de los resultados de la investigación.



Figura 30

OPENVAS

Herramienta de seguridad informática conocida por tratarse de una alternativa de código abierto a la herramienta comercial Nessus.

Al igual que esta, permite realizar las siguientes acciones a los profesionales de seguridad:

- **Escaneo de vulnerabilidades.** Ofrece la posibilidad de realizar un escaneo de seguridad automatizado en sistemas y redes para identificar vulnerabilidades conocidas.
- **Análisis de configuración.** También puede efectuar un análisis para detectar configuraciones inseguras o erróneas.

- **Detección de fallos de seguridad.** Identifica defectos de seguridad en servicios y aplicaciones, como problemas de autenticación, configuraciones de permisos incorrectas, errores de configuración SSL/TLS, etc.
- **Clasificación de la gravedad de las vulnerabilidades.** Las vulnerabilidades se clasifican en función de su gravedad, lo que permite a los administradores priorizar las acciones.
- **Generación de informes.** Tras realizar los escaneos, la herramienta genera informes donde se detallan las vulnerabilidades detectadas y las acciones recomendadas para solventar los problemas.



Figura 31

QUALYS

Herramienta que ofrece soluciones de cara a la gestión de vulnerabilidades a partir de un modelo de negocio conocido como *Software-as-a-Service*. Así, Qualys ofrece un servicio de desarrollo de *software* a medida especializado en la gestión y detección de fallos de seguridad. Actualmente, presta sus servicios a más de 10 300 clientes de todo el mundo, tratándose de una empresa tan competitiva que sus acciones se cotizan en el mercado Nasdaq. El uso de esta herramienta requiere la adquisición de una licencia comercial.

El proceso de Qualys es el siguiente:

- **Se ejecuta un escaneo de vulnerabilidades** que consiste en identificar los dispositivos conectados a una red y en verificar qué posibles fallos de seguridad presentan su *software* y *hardware*.
- **Se organizan las vulnerabilidades detectadas.** En este sentido, se establecen categorías que pueden asociarse a los siguientes aspectos:
 - La ubicación.
 - El *software*.
 - El sistema operativo.
 - El nivel de prioridad.
- **Se almacenan los datos relativos a dichas vulnerabilidades**, de manera que permanezcan accesibles para los investigadores y los motores de escaneo.
- **Se elabora un reporte**, es decir, se comunican los hallazgos al equipo para que conozcan los fallos de seguridad presentes en el sistema. Recurriendo a esta información, otros miembros del equipo remediarán esos fallos mediante configuraciones o el desarrollo de parches de seguridad.

- Finalmente, se aborda la **fase de verificación**, cuyo propósito consiste en poner a prueba las soluciones desarrolladas por el equipo de seguridad.



Figura 32

NIKTO

Herramienta ampliamente utilizada en el ámbito de la ciberseguridad, especialmente de cara al escaneo de vulnerabilidades en servidores web con el objeto de identificar posibles problemas de seguridad. Se trata de una herramienta de código abierto que ha ido adquiriendo peso progresivamente, siendo muy empleada por administradores de sistemas y profesionales de la seguridad.

Se puede utilizar para escanear cualquier servidor web (Apache, Nginx, Lighttpd, Litespeed, etc.), de momento más de 1250 (y sigue creciendo), frente a más de 6700 vulnerabilidades conocidas y verificaciones de versión. Sus principales funciones son:

- Escaneos exhaustivos del servidor web objetivo y de certificados SSL.
- Capacidad para escanear distintos puertos en un sistema con varios servidores web en ejecución y opción de escanear a través de un *proxy* y con autenticación HTTP.
- Escaneo de línea de comandos, permitiendo a los usuarios y las usuarias personalizar este proceso según sus necesidades específicas.
- Detección de vulnerabilidades recurriendo a una base de datos de *plugins* y firmas de vulnerabilidades conocidas que se actualiza regularmente.
- Análisis en busca de problemas relacionados con la configuración, como directorios de índice abiertos.
- Capacidad para definir el tiempo máximo de escaneo, excluir ciertos tipos de escaneos y visualizar encabezados de informes inusuales.
- Generación de informes detallados después de cada escaneo.



Nikto

Figura 33

NMAP

Herramienta de código abierto para el escaneo de redes creada originalmente para Linux, si bien actualmente es multiplataforma, convirtiéndose en una de las soluciones más populares y respetadas en la comunidad de ciberseguridad. Nmap se emplea para identificar *hosts* y servicios en una red, así como para mapear su topología.

Esta herramienta permite una amplia gama de escaneos, entre otros:

- **Escaneo de puertos**, identificando aquellos que permanecen abiertos en un *host*.
- **Identificación de servicios**, determinando cuáles están funcionando en los puertos abiertos.
- **Detección de sistemas operativos**, identificando el del *host* en función de las respuestas a los paquetes enviados.
- **Escaneos específicos para detectar vulnerabilidades conocidas** en servicios y sistemas.
- **Escaneo de redes** en rangos de direcciones IP o subredes completas.
- **Escaneo sigiloso**, minimizando el ruido de red.



Figura 34